

Task 16: Selecting Rows and Columns with loc and iloc

Task description: Practice selecting rows and columns using Pandas **loc[]** and **iloc[]** and understand label-based versus position-based selection.

Objective: Develop accurate DataFrame selection skills.

Tools: Python, Pandas, Jupyter Notebook.

1. Why Row and Column Selection Is Important

A real DataFrame may contain thousands or millions of records and many columns. In analysis, we rarely need everything at once. Selection allows us to retrieve exactly the records and fields required for a task. For example, we may need only the names and average marks of students whose average is above 80.

2. DataFrame Index and Columns

Every DataFrame has an **index** for rows and labels for columns. In this project, the row labels are S001, S002, ..., S008. The column labels include Name, Age, Python_Marks, SQL_Marks, Attendance, and Average. Understanding these labels is essential for using **loc[]** correctly.

3. The Main Difference

The most important rule in this task is: **loc[] selects by labels, while iloc[] selects by integer positions.** If a row is labeled S001, `df.loc['S001']` selects it. The same row is at position 0, so `df.iloc[0]` also selects it in this dataset.

4. loc[] Syntax

The general form is `df.loc[row_selector, column_selector]`. The row selector can be a single label, a list of labels, a label range, or a Boolean condition. The column selector can be a column name, a list of column names, or `:` for all columns.

5. loc Example 1 — One Complete Row

`df.loc['S001']` returns the complete row with label S001. This is useful when you know the exact row label you want.

6. loc Example 2 — Multiple Rows

`df.loc[['S002', 'S004', 'S006']]` selects several rows by providing a list of labels. This is useful when the required records are not consecutive.

7. loc Example 3 — One Cell

`df.loc['S003', 'Python_Marks']` selects the value at the intersection of row label S003 and column label Python_Marks. This is the clearest way to retrieve a known cell when you know its labels.

8. loc Example 4 — Multiple Rows and One Column

`df.loc[['S001', 'S003', 'S005'], 'Python_Marks']` selects Python marks for three labeled students. The result is a Series because only one column is requested.

9. loc Example 5 — Multiple Rows and Multiple Columns

`df.loc[['S002', 'S004', 'S006'], ['Name', 'SQL_Marks', 'Attendance']]` selects specific students and specific named columns. This is useful for creating a focused analysis table.

10. loc Example 6 — Label Range

`df.loc['S002':'S005', ['Name', 'Average']]` selects a range of row labels and two named columns. With `loc[]`, label-based slicing includes the ending label when the labels are ordered.

11. loc Example 7 — All Rows and Selected Columns

The colon `:` means all rows. Therefore, `df.loc[:, ['Name', 'Average']]` selects every row but only the Name and Average columns.

12. loc Example 8 — Conditional Selection

`df.loc[df['Average'] >= 80, ['Name', 'Average']]` first creates a Boolean condition and then returns only the requested columns. This is one of the most useful real-world patterns in Pandas.

13. loc Example 9 — Multiple Conditions

Use `&` for AND and `|` for OR. Example: `df.loc[(df['Average'] >= 80) & (df['Attendance'] >= 85), ['Name', 'Average', 'Attendance']]`. Parentheses around each condition are important.

14. iloc[] Syntax

The general form is `df.iloc[row_position, column_position]`. Positions are integer values starting at 0. The first row is position 0, the second row is position 1, and so on.

15. iloc Example 1 — First Row

`df.iloc[0]` returns the first row. It does not care what the row label is; it uses its numerical position.

16. iloc Example 2 — Multiple Row Positions

`df.iloc[[1,3,5]]` returns the second, fourth, and sixth rows because positions start from 0.

17. iloc Example 3 — One Cell

`df.iloc[2,3]` selects the value at row position 2 and column position 3. This is useful when you know the numerical coordinates of the required value.

18. iloc Example 4 — Selected Positions

`df.iloc[1:5, [1,3,4]]` selects rows from position 1 through 4 and columns at positions 1, 3, and 4. Positional slicing follows Python's normal slicing rule: the stop position is excluded.

19. iloc Example 5 — Rectangular Block

`df.iloc[2:6, 1:5]` selects a block of rows and columns. It is similar to selecting a rectangular range in a spreadsheet by position.

20. iloc Example 6 — Negative Positions

`df.iloc[-2:]` selects the last two rows. Negative indexing is useful when you need the end of a dataset without knowing its exact length.

21. iloc Example 7 — First Rows and Columns

`df.iloc[:3, :4]` selects the first three rows and first four columns. The stop values are excluded, so rows 0,1,2 and columns 0,1,2,3 are returned.

22. iloc Example 8 — Step Selection

`df.iloc[::2]` selects every second row. The step value is 2. This demonstrates that normal Python slicing rules can be used with `iloc[]`.

23. loc vs iloc — Detailed Comparison

loc[]: uses row labels and column names; best when labels are meaningful; label slicing includes the stop label.

iloc[]: uses integer positions; best when the selection is position-driven; positional slicing excludes the stop position.

24. Choosing the Correct Method

Choose **loc[]** when you can describe the selection using labels or business conditions, such as 'students with average above 80'. Choose **iloc[]** when the requirement is based on physical position, such as 'first three rows' or 'columns 1 to 4'.

25. Common Mistake — Confusing Labels and Positions

If the index labels are S001, S002, etc., `df.loc[0]` will not mean the first row because 0 is not a label. `df.iloc[0]` means the first row because `iloc` uses positions.

26. Common Mistake — Using Column Names with iloc

This is incorrect: `df.iloc[:, 'Name']`. `iloc[]` requires integer positions. Use `df.loc[:, 'Name']` when selecting by a column name.

27. Common Mistake — Boolean Operators

For multiple conditions use `&` and `|`, not Python's `and` and `or` for Series conditions. Put each condition in parentheses.

28. Best Practices

Use `loc[]` for readable, label-based analysis. Use `iloc[]` for position-based operations. Verify the index and columns before selecting. Prefer explicit column names when readability matters, and test selection expressions on a small sample before applying them to large datasets.

29. Practical Student Analysis

The project uses student marks and attendance to demonstrate selection. For example, you can select high-performing students, inspect their attendance, retrieve particular students' marks, or create a smaller table containing only Name, Average, and Attendance.

30. Complete Practical Python Program

```
import pandas as pd

# Load the dataset and keep the custom student labels as the index
df = pd.read_csv("student_selection.csv", index_col=0)

print("FULL DATAFRAME")
print(df)

# LOC[] — LABEL-BASED SELECTION

# 1. One complete row by label
print("\nLOC 1")
print(df.loc["S001"])

# 2. Several complete rows by labels
print("\nLOC 2")
print(df.loc[["S002", "S004", "S006"]])

# 3. One row + one column
```

```

print("\nLOC 3")
print(df.loc["S003", "Python_Marks"])

# 4. Several rows + one column
print("\nLOC 4")
print(df.loc[["S001", "S003", "S005"], "Python_Marks"])

# 5. Several rows + several named columns
print("\nLOC 5")
print(df.loc[
    ["S002", "S004", "S006"],
    ["Name", "SQL_Marks", "Attendance"]
])

# 6. Label range + selected columns
print("\nLOC 6")
print(df.loc[
    "S002":"S005",
    ["Name", "Python_Marks", "Average"]
])

# 7. All rows + selected columns
print("\nLOC 7")
print(df.loc[:, ["Name", "Average"]])

# 8. Conditional selection
print("\nLOC 8")
print(df.loc[df["Average"] >= 80, ["Name", "Average"]])

# 9. Multiple conditions
print("\nLOC 9")
print(df.loc[
    (df["Average"] >= 80) & (df["Attendance"] >= 85),
    ["Name", "Average", "Attendance"]
])

# ILOC[] - POSITION-BASED SELECTION

# 1. First row
print("\nILOC 1")
print(df.iloc[0])

# 2. Several rows by positions
print("\nILOC 2")
print(df.iloc[[1, 3, 5]])

# 3. One cell: row position 2, column position 3
print("\nILOC 3")
print(df.iloc[2, 3])

# 4. Row positions + selected column positions
print("\nILOC 4")
print(df.iloc[1:5, [1, 3, 4]])

# 5. Rectangular block
print("\nILOC 5")
print(df.iloc[2:6, 1:5])

# 6. Last two rows
print("\nILOC 6")
print(df.iloc[-2:])

# 7. First three rows and first four columns
print("\nILOC 7")
print(df.iloc[:3, :4])

# 8. Every second row
print("\nILOC 8")
print(df.iloc[::2])

# COMPARISON

print("\nKEY RULE")
print("loc[] -> labels")
print("iloc[] -> integer positions")

# Practical result
print("\nHIGH-PERFORMING STUDENTS")
print(df.loc[
    df["Average"] >= 85,

```

```
["Name", "Python_Marks", "SQL_Marks", "Average"]  
})
```

31. Quick Reference Table

Feature	loc[]	iloc[]
Selection type	Label-based	Position-based
Row selector	Labels	Integers
Column selector	Column names	Integer positions
First row	df.loc['S001']	df.iloc[0]
One column	df.loc[:, 'Name']	df.iloc[:, 0]
Range	Label range	Position range
Stop in slice	Included for labels	Excluded for positions
Best use	Meaningful labels / conditions	Numerical positions

32. Interview Questions and Answers

Q1. What is loc[]?

A: A Pandas indexer used for label-based row and column selection.

Q2. What is iloc[]?

A: A Pandas indexer used for integer-position-based selection.

Q3. When should you use loc[]?

A: When selecting using row labels, column names, or Boolean conditions.

Q4. When should you use iloc[]?

A: When selecting by numerical row and column positions.

Q5. Can loc[] use column names?

A: Yes.

Q6. Can iloc[] use column names?

A: No. It requires integer positions.

Q7. Does loc label slicing include the end label?

A: Yes, label-based slicing is inclusive of the ending label when applicable.

Q8. Does iloc positional slicing include the stop position?

A: No. It follows normal Python slicing and excludes the stop position.